

Subprograms: Lambdas, Closures, Generators, and Coroutines

Programming Languages

Millersville University

Outline

- Lambda Functions
- Closures
- Re-entrant Subprograms
 - Generators
 - Coroutines

A decorative graphic on the left side of the slide. It features a series of yellow dashed lines forming a partial arc, starting from the top left and curving downwards. A solid blue circle is positioned at the bottom right of the white oval, partially overlapping its edge.

Lambda Functions

Lambda Functions

- **Lambda Function**, also called

- function literal
- lambda abstraction
- anonymous function
- lambda expression

- Originate from *Alonzo Church* and his invention of Lambda Calculus around 1936

(all functions were anonymous)

$$(\lambda x. x^2 - 2x + 5) 2$$

*In programming languages since **1958***

Lambda Function Tour

Lisp

```
(lambda () sexpr)  
(lambda (x y) (+ x y))
```

JavaScript

```
() => expr  
() => { stmts... }  
(x, y) => x + y
```

Lambda Function Tour

OCaml

```
fun () -> ...
```

```
fun x y -> x + y
```

Java

```
() -> expr
```

```
() -> { stmts... }
```

```
(x, y) -> x + y
```

```
(x, y) -> { return x + y; }
```

```
(double x, double y) -> { return x + y; }
```

Lambda Function Tour

Python

`lambda: expr`

`lambda x, y: x + y`

Swift

`{ () in expr }`

`{ () in stmts... }`

`{ x, y in x + y }`

`{ (x: int, y: int) -> int in return x + y }`

`{ $0 + $1 }`

Lambda Function Tour

C++

```
[]() { stmts... }
```

```
[](auto x, auto y) { return x + y; }
```

```
[](int x, int y) -> int { return x + y; }
```

Ruby

```
lambda { expr }
```

```
lambda { stmts... }
```

```
lambda { |x, y| x + y }
```

```
-> (x, y) { x + y }
```


A decorative graphic on the left side of the slide. It consists of a series of yellow dashed lines of varying lengths and thicknesses, arranged in a curved, semi-circular pattern. At the bottom right of this pattern is a small, solid blue circle.

Closures

Closures

- A *closure* is a subprogram AND *referencing environment* for where it was defined
 - Key difference from lambdas: can access *non-local vars*
- Most often written as a lambda with no syntactic difference
 - Lisp
 - Java
 - JavaScript
 - OCaml
 - Python
 - Swift
 - Ruby
- Exception: C++

Closure Example

JavaScript

$x \Rightarrow y \Rightarrow x + y$

$x \Rightarrow \{ \text{return } y \Rightarrow x + y \}$

^

OCaml

```
let multiply n lst =  
  List.map (fun x -> n * x) lst
```

^

Closure Example

C++

```
namespace r = std::ranges;
std::vector<int>
multiply (int n, std::vector<int> list) {
    r::for_each (list, [n] (int& value) {
        return value * n; //^ explicit copy of n
    });
    return list;
}
```

Closure Example

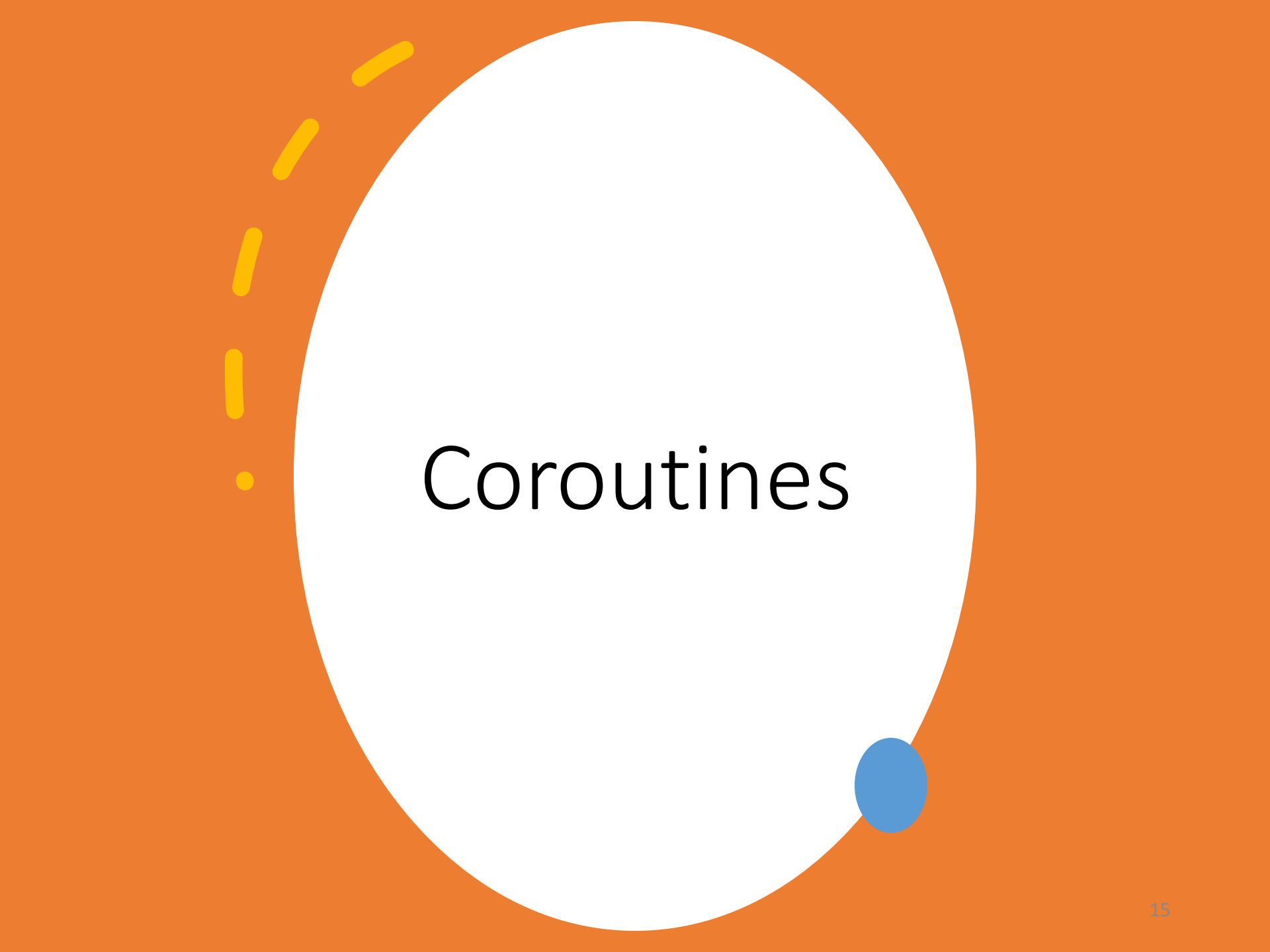
C++

```
namespace r = std::ranges;
std::vector<int>
multiply (int n, std::vector<int> list) {
    r::for_each (list, [=] (int& value) {
        return value * n; //^ all non-locals are
    });                  // passed by value
    return list;
}
```

Closure Example

C++

```
namespace r = std::ranges;
std::vector<int>
multiply (int n, std::vector<int> list) {
    r::for_each (list, [&] (int& value) {
        return value * n; //^ all non-locals are
    });                  // passed by reference
    return list;
}
```



Coroutines

Subprogram Definition

A subprogram has...

- Exactly one entry point
- One or more exit points

Subprogram Definition

A subprogram has...

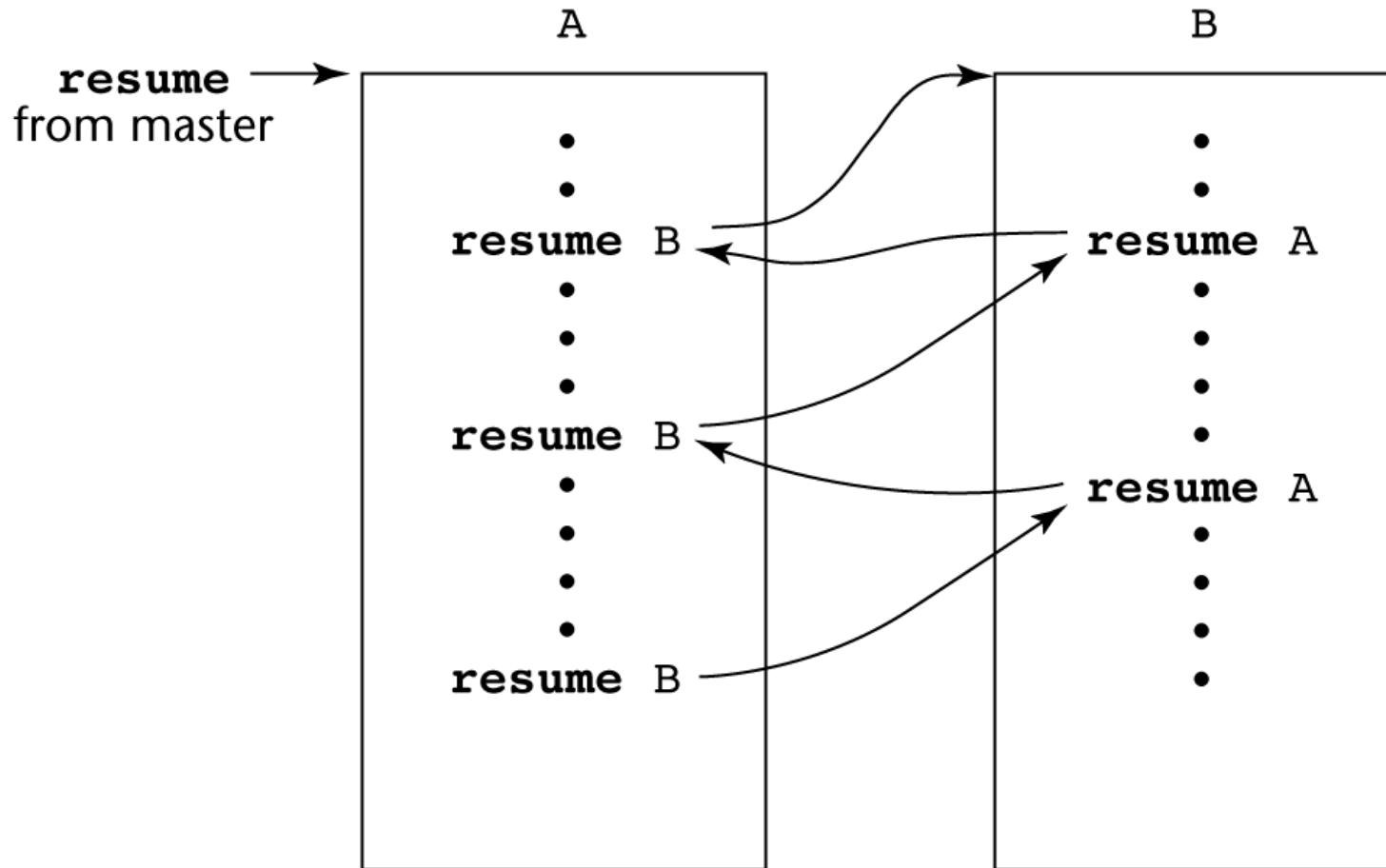
- Exactly one entry point
 - The beginning of the function!
- One or more exit points
 - Can have return statements throughout
 - Exceptional control flow (later in course)



Coroutines

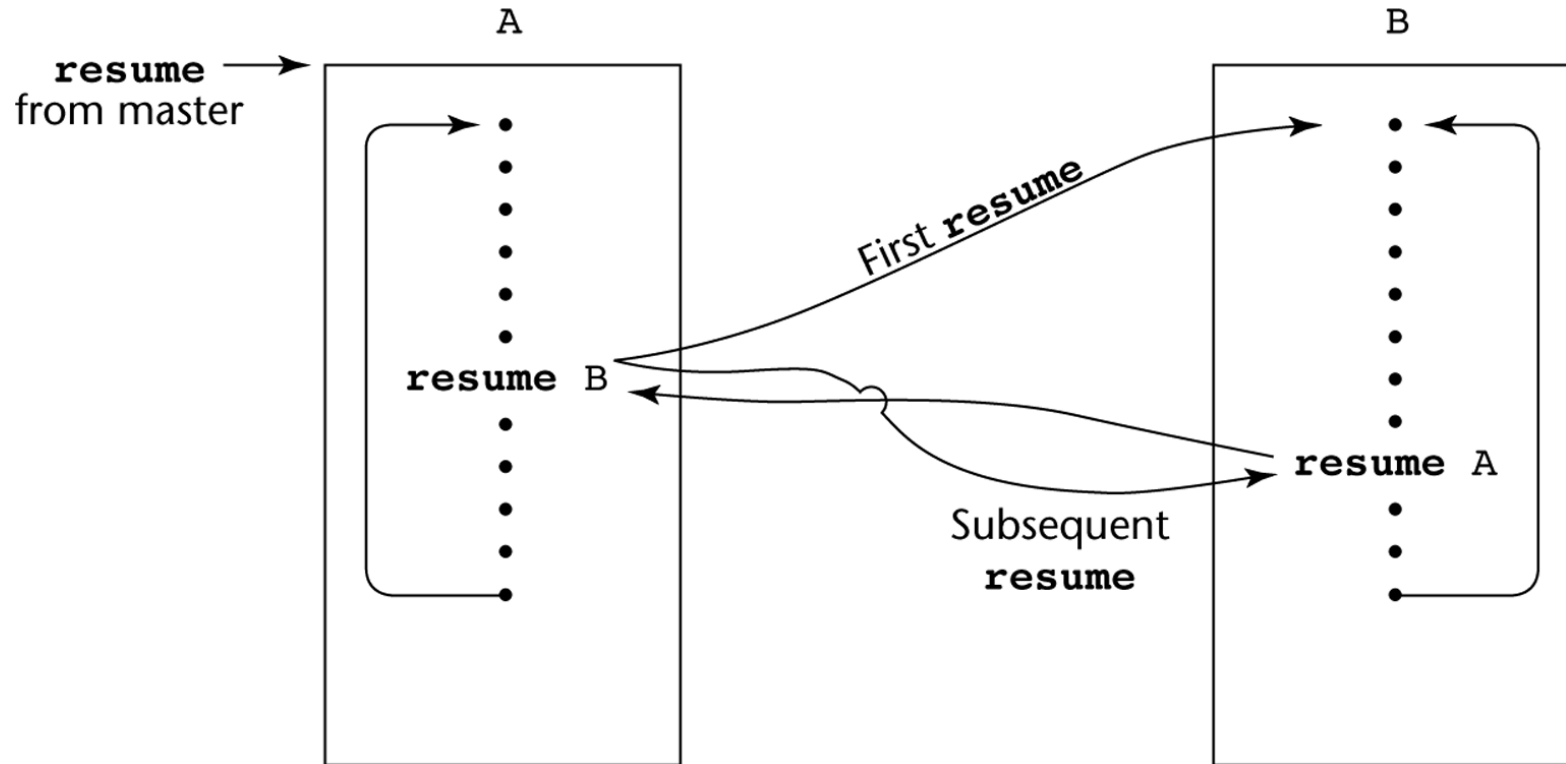
- A *coroutine* is a subprogram that has multiple entry points
- Also called *symmetric control*
Caller and callee coroutines are on a more equal basis
- A coroutine call is named a *resume*
 - First resume of a coroutine is to its beginning
 - Subsequent calls pickup where left off
- Coroutines provide *quasi-concurrent execution*
 - Good for concurrent programming
 - Execution is interleaved, but not overlapped

Coroutines Illustrated: Possible Execution Controls



(a)

Coroutines Illustrated: Possible Execution Controls with Loops



Coroutines: Language Support

- C++ (since C++20)
- C#
- D
- F#
- Go
- JavaScript
- Julia
- Lua
- PHP
- Prolog
- Python
- Ruby
- Rust
- Scheme (Lisp-like)

Coroutines Example

```
def match(pattern):  
    print('Looking for ' + pattern)  
    try:  
        while True:  
            s = (yield)  
            if pattern in s:  
                print(s)  
    except GeneratorExit:  
        print('Done.')
```

Coroutines Example

```
>>> matcher = match('hello')
>>> next(matcher)
    Looking for hello
>>> matcher.send('hello there')
    hello there
>>> matcher.send('goodbye now')
>>> matcher.send('Othello is a great play')
    Othello is a great play
>>> matcher.close()
    Done.
```

```
def match(pattern):
    print('Looking for ' + pattern)
    try:
        while True:
            s = (yield)
            if pattern in s:
                print(s)
    except GeneratorExit:
        print('Done.')
```

Generators

- Generators are an example of coroutines
- Generators can give us different values each invocation time
- Values are not computed when the generator is created, but when they are requested!

Generators

Goal

- Don't want to exit the subprogram – simply “pause” it in some way
- Achieved by **not** using **return**
- Define a new control flow keyword!

Generators

- **yield** values
- Yield “pauses” execution of the subprogram
- When we invoke subprogram we resume from where we left off

Python:

```
def first_three():  
    yield 1  
    yield 2  
    yield 3
```

Generators

```
def ones():  
    while True:  
        yield 1
```

```
def natural():  
    x = 0  
    while True:  
        yield x  
        x += 1
```

```
gen = natural()  
next(gen)  
next(gen)  
next(gen)  
next(gen)
```

Generators

Exercise

```
def fibonacci():
```

Recursive Generators

```
from os import listdir
from os.path import isfile, join

def print_files(dir):
    for file in listdir(dir):
        full_path = join(dir, file)
        if isfile(full_path):
            yield full_path
        else:
            yield from print_files(full_path)
```

Attribution

Slides originally developed by William Killian,
modified by Stephanie Schwartz
modified by Gary Zoppetti

Sources include Robert Sebesta's [Concepts of Programming Languages](#)